

PARADIGMAS DE LA PROGRAMACIÓN

Tarea 2

Ficha Técnica del Sistema de Gestión de Rutinas Deportivas



Autor:

Christopher Torrejón ^(a, b)

^(a) Ingeniería Civil Industrial. Facultad de Ingeniería, Universidad Andrés Bello. Chile

^(b) Ingeniería Civil Informática. Facultad de Ingeniería, Universidad Andrés Bello. Chile

c.torrejnallende@uandresbello.edu

21.774.845-3

Paradigmas de la Programación **[8025]**

Profesor encargado:

Juan Felipe Calderón Maureira

Mayo 28, 2026. Valparaíso, Viña del Mar

Tabla de contenido

1.	Introducción Técnica	1
2.	Arquitectura General del Sistema	1
2.1.	Estructura del Backend.....	2
2.2.	Estructura del Frontend.....	3
2.3.	Comunicación entre Componentes.....	4
3.	Reglas de Negocio Implementadas	6
4.	Conclusión Técnica	7

1. Introducción Técnica

El propósito del presente documento consiste en complementar la información expuesta dentro del informe principal mediante una descripción más detallada del funcionamiento técnico y lógico del sistema desarrollado. Para ello, se profundiza en la arquitectura implementada, las reglas de negocio consideradas durante el desarrollo y la organización estructural de los distintos componentes que conforman la aplicación.

Asimismo, el documento busca descomponer y explicar las responsabilidades asociadas tanto al backend como al frontend, permitiendo comprender la manera en que ambos bloques interactúan entre sí para garantizar el correcto funcionamiento del programa. En este contexto, se detallan las principales clases implementadas, sus dependencias y el rol que cumplen dentro de la solución desarrollada.

Finalmente, esta ficha técnica tiene como objetivo facilitar la correcta interpretación del diagrama de clases UML presentado en el informe principal, proporcionando una visión más específica sobre la lógica interna del sistema, las relaciones existentes entre componentes y las decisiones técnicas adoptadas durante el proceso de implementación.

2. Arquitectura General del Sistema

La arquitectura del sistema fue desarrollada mediante una separación entre frontend y backend, permitiendo distribuir adecuadamente las responsabilidades relacionadas con la lógica de negocio y la interacción gráfica con el usuario. Esta organización favorece la modularidad, el mantenimiento del código y la correcta administración de los datos utilizados por la aplicación.

Dentro del backend se encuentran las clases responsables de representar las entidades principales del sistema, administrar las estructuras de datos y controlar las reglas de negocio implementadas. Entre estas destacan las clases *Ejercicio*, *Rutina*, *ColeccionEjercicio*, *ColeccionRutina*, *Orquestador*, *ProcesamientoDatos* y *LecturaArchivo*, cada una encargada de funciones específicas relacionadas con persistencia, validación y procesamiento de información.

Por otra parte, el frontend se encuentra compuesto por las distintas ventanas desarrolladas mediante Java Swing, las cuales permiten al usuario interactuar visualmente con el sistema mediante programación orientada a eventos. Estas ventanas encapsulan funcionalidades específicas asociadas a carga de archivos, generación de rutinas, búsqueda de información y visualización de resultados.

La comunicación entre ambos bloques se realiza mediante el intercambio controlado de estructuras de datos y solicitudes generadas desde eventos asociados a la interfaz gráfica. Gracias a esta arquitectura, el sistema logra mantener separadas las responsabilidades visuales y lógicas de la aplicación, evitando exponer directamente la manipulación interna de los datos al usuario final.

2.1. Estructura del Backend

La clase *Ejercicio* se encarga de representar computacionalmente la estructura general de un ejercicio dentro del sistema. Para ello, posee los atributos *codigoidentificador*, *nombre*, *tipoEjercicio*, *nivelIntensidad*, *tiempoEstimado*, *descripcion* y *ultimaSemana*, permitiendo almacenar las características fundamentales asociadas a cada ejercicio disponible en la aplicación.

A partir de esta clase base nacen dos subclases especializadas: *EjercicioCardiovascular* y *EjercicioFuerza*, las cuales extienden la funcionalidad general de *Ejercicio* incorporando atributos específicos según el tipo de entrenamiento. En el caso de *EjercicioCardiovascular*, se añade el atributo *tipoMaquina*, mientras que *EjercicioFuerza* incorpora el atributo *grupoMuscular*. Gracias a esta estructura, el sistema aplica herencia para reutilizar atributos y comportamientos comunes, manteniendo al mismo tiempo características particulares según cada categoría de ejercicio.

La clase *Ejercicio* cuenta con métodos getters para todos sus atributos y setters para todos ellos excepto *codigoidentificador*, ya que este corresponde a un identificador único que no debe modificarse posteriormente. Asimismo, incorpora un método polimórfico denominado *mostrar()*, utilizado para representar visualmente la información asociada a cada ejercicio dentro del sistema.

Por otra parte, la clase *Rutina* tiene como objetivo almacenar los ejercicios asignados a un cliente según determinadas restricciones definidas por el usuario, tales como cantidad de ejercicios por tipo, semana de entrenamiento y nivel de intensidad requerido. Esta clase posee como atributos un *ArrayList<Ejercicio>* denominado *ejercicios*, además de *cliente*, *tiempoTotal* y *ultimaSemana*. De igual manera, incorpora sus respectivos métodos getters y setters, junto a métodos adicionales destinados a mostrar características asociadas a la rutina generada, como cantidades de ejercicios por tipo, intensidad y tiempo total acumulado.

La clase *LecturaArchivo* fue desarrollada exclusivamente para realizar la lectura de archivos externos utilizados por el sistema, permitiendo obtener de manera íntegra la información contenida dentro de ellos. Complementariamente, la clase *ProcesamientoDatos* se encarga de interpretar dichos datos y asignarlos correctamente a las estructuras y objetos correspondientes dentro del programa.

Asimismo, la clase *ColeccionEjercicio* representa un contenedor principal basado en un *ArrayList<Ejercicio>*, donde se almacenan todos los ejercicios disponibles dentro del sistema. Además de sus respectivos métodos getters y setters, esta clase incorpora funcionalidades orientadas al análisis y procesamiento de datos, tales como cálculo de ejercicios por tipo, nivel de intensidad y tiempo total disponible, junto a métodos encargados de mostrar estas estadísticas de manera organizada.

De manera similar, la clase *ColeccionRutina* funciona como una estructura de almacenamiento para todas las rutinas correctamente generadas por el sistema. Además de administrar dinámicamente dichas rutinas, esta clase incorpora métodos orientados a mostrar información asociada a clientes existentes y características relevantes relacionadas con las rutinas almacenadas.

Finalmente, la clase *Orquestador* representa el componente central del backend y una de las piezas más importantes de toda la arquitectura implementada. Su principal responsabilidad consiste en coordinar la lógica necesaria para garantizar que las rutinas sean creadas correctamente según las restricciones establecidas por el sistema. Para ello, utiliza distintos métodos especializados que permiten validar y controlar cada etapa del proceso de generación de rutinas.

En primer lugar, mediante el método *ejerciciosDisponibles()*, el sistema discrimina aquellos ejercicios que no pueden ser utilizados según las restricciones asociadas al cliente y la semana correspondiente. Posteriormente, el método *factibilidad()* verifica si existe una cantidad suficiente de ejercicios disponibles para satisfacer los requerimientos solicitados por el usuario según tipo e intensidad. Luego, utilizando el método *filtro()*, se construye correctamente la nueva rutina incorporando los ejercicios válidos seleccionados. Finalmente, mediante el método *existeRutina()*, se verifica que la rutina generada no se encuentre duplicada para un mismo cliente dentro de una misma semana. Todo este proceso es centralizado a través del método *crearRutina()*, encargado de coordinar todas las validaciones y generar correctamente la rutina final o indicar, en caso contrario, que no es posible satisfacer los requerimientos solicitados.

2.2. Estructura del Frontend

La estructura del frontend fue desarrollada mediante distintas ventanas implementadas utilizando Java Swing, permitiendo construir una interfaz gráfica interactiva basada en programación orientada a eventos. Cada ventana fue diseñada con una responsabilidad específica dentro del flujo general del sistema, permitiendo organizar adecuadamente las funcionalidades disponibles para el usuario.

La *VentanaPrincipal* representa el núcleo central del frontend, ya que todas las interacciones iniciales del usuario comienzan desde esta interfaz. Dentro de ella, el usuario dispone inicialmente de dos opciones principales: cerrar el programa o cargar los datos necesarios para el funcionamiento de la aplicación. Asimismo, existen las opciones asociadas a generación y búsqueda de rutinas; sin embargo, estas permanecen deshabilitadas hasta que el sistema detecta una carga correcta de los datos.

Cuando el usuario selecciona la opción de carga de archivo, se abre la *VentanaArchivo*, cuya responsabilidad consiste en permitir el ingreso y procesamiento de los datos externos requeridos por el sistema. Desde esta ventana, el usuario puede regresar a la ventana principal o ejecutar el proceso de carga correspondiente. Si los datos son procesados correctamente, el sistema muestra información asociada al conjunto cargado, tales como cantidad de ejercicios disponibles por tipo, niveles de intensidad existentes y tiempo total disponible. Una vez completada correctamente esta etapa, la opción de carga queda deshabilitada con el objetivo de preservar la integridad y consistencia de la información almacenada.

Posteriormente, al regresar a la *VentanaPrincipal*, se habilitan las funcionalidades de generación y búsqueda de rutinas.

La opción “Generar Rutina” abre la *VentanaGenerarRutina*, donde el usuario debe ingresar distintos parámetros necesarios para la creación de una nueva rutina deportiva. Entre estos datos se encuentran el identificador del cliente, semana de entrenamiento, cantidad de ejercicios requeridos por tipo (Cardiovascular y Fuerza) y el nivel de intensidad asociado a la rutina completa. Mediante la comunicación establecida con la clase *Orquestador*, el sistema verifica automáticamente que las restricciones definidas sean cumplidas, permitiendo crear correctamente la rutina o informar al usuario sobre posibles errores o inconsistencias detectadas durante el proceso.

Por otra parte, la opción “Buscar Rutina” permite acceder a la *VentanaBuscarRutina*, cuya finalidad consiste en localizar y explorar rutinas específicas asociadas a clientes determinados. Para ello, inicialmente se solicita el identificador del cliente. El sistema valida previamente que dicho cliente exista y que corresponda a un identificador válido antes de habilitar la selección de rutinas asociadas. Una vez seleccionada la rutina deseada, se abre la *VentanaRevisarRutina*, donde el usuario puede recorrer individualmente todos los ejercicios pertenecientes a dicha rutina.

Dentro de esta ventana, el sistema incorpora mecanismos de control orientados a preservar una navegación coherente. Por ejemplo, al encontrarse en el primer ejercicio, el botón “Volver” permanece deshabilitado para evitar desplazamientos inválidos dentro de la colección de ejercicios. De igual manera, al alcanzar el último ejercicio de la rutina, el botón “Siguiente” modifica dinámicamente su texto a “Resumen de Rutina”, permitiendo acceder finalmente a la *VentanaResumenRutina*.

Esta última ventana tiene como propósito mostrar las características generales asociadas a la rutina seleccionada, incluyendo información relevante relacionada con cantidad de ejercicios, distribución por tipo, intensidad y tiempo total acumulado. Una vez finalizada la revisión correspondiente, el usuario retorna nuevamente a la *VentanaPrincipal*, pudiendo repetir el ciclo de utilización del sistema según las funcionalidades que requiera ejecutar.

2.3. Comunicación entre Componentes

La comunicación entre frontend y backend dentro del sistema se encuentra basada principalmente en tres conexiones fundamentales, las cuales permiten coordinar correctamente la interacción entre la interfaz gráfica y la lógica interna del programa.

La primera conexión ocurre entre la *VentanaPrincipal* y las colecciones principales del sistema, correspondientes a *ColeccionEjercicio* y *ColeccionRutina*. Esta relación permite que la interfaz gráfica pueda solicitar acciones y acceder indirectamente a la información almacenada dentro del backend. Bajo esta lógica, el frontend actúa principalmente como una capa visual encargada de detectar eventos y solicitudes realizadas por el usuario, mientras que el backend se responsabiliza del procesamiento lógico y manipulación de los datos asociados a dichas acciones.

La segunda comunicación relevante se produce durante el proceso de carga de archivos. En este caso, la *VentanaArchivo* solicita la carga de los datos necesarios para el funcionamiento del sistema. Posteriormente, mediante la clase *ProcesamientoDatos*, y utilizando como apoyo la clase *LecturaArchivo*, el programa realiza la lectura e interpretación de la información contenida dentro del archivo externo. Finalmente, los datos obtenidos son asignados correctamente a las estructuras y objetos correspondientes dentro del backend, permitiendo construir dinámicamente los ejercicios disponibles utilizados posteriormente por el sistema.

La tercera conexión, y probablemente la más importante dentro de toda la arquitectura implementada, ocurre durante el proceso de generación de rutinas. En esta etapa, el usuario ingresa los parámetros requeridos desde la *VentanaGenerarRutina*, tales como identificador del cliente, semana de entrenamiento, cantidad de ejercicios y nivel de intensidad solicitado. Posteriormente, la información ingresada es enviada a la clase *Orquestador*, la cual se encarga de gestionar toda la persistencia lógica asociada a la creación de la rutina.

Dentro de este proceso, el *Orquestador* valida restricciones, verifica factibilidad, discrimina ejercicios no permitidos y controla que no existan rutinas duplicadas para un mismo cliente dentro de una misma semana. Como resultado, el sistema puede generar correctamente una rutina válida o, en caso contrario, informar al usuario que los parámetros ingresados son incorrectos o no factibles según las restricciones existentes.

Gracias a esta arquitectura de comunicación, el sistema logra mantener una separación clara entre interfaz gráfica y lógica de negocio, evitando que el frontend manipule directamente las estructuras internas del programa y favoreciendo una administración más segura, organizada y mantenible de la información.

3. Reglas de Negocio Implementadas

El sistema desarrollado incorpora un conjunto de reglas de negocio orientadas a garantizar la coherencia lógica de las rutinas generadas y preservar la integridad de la información administrada por la aplicación. Estas restricciones fueron implementadas tanto a partir de requerimientos explícitos definidos en el enunciado como mediante validaciones adicionales incorporadas durante el proceso de diseño y desarrollo del sistema.

Una de las principales reglas implementadas corresponde a la imposibilidad de asignar los mismos ejercicios a un mismo cliente durante dos semanas consecutivas. Para satisfacer esta restricción, el sistema discrimina automáticamente aquellos ejercicios utilizados previamente por el cliente durante la semana inmediatamente anterior, evitando repeticiones que puedan afectar la variabilidad de las rutinas generadas.

Asimismo, el sistema impide la creación de rutinas duplicadas para un mismo cliente dentro de una misma semana. Esta validación fue considerada necesaria para mantener la consistencia lógica de la aplicación y evitar redundancia en la información almacenada dentro de las colecciones principales del programa.

De manera complementaria, antes de generar una nueva rutina, el sistema realiza una validación de factibilidad orientada a verificar si existe una cantidad suficiente de ejercicios disponibles según el tipo solicitado y el nivel de intensidad seleccionado por el usuario. Gracias a este proceso, únicamente se generan rutinas que puedan satisfacer correctamente los parámetros definidos durante la solicitud.

Por otra parte, se incorporaron validaciones asociadas al ingreso de datos por parte del usuario. Entre ellas destacan controles sobre formatos numéricos, restricciones sobre valores negativos y verificación de campos obligatorios, permitiendo evitar inconsistencias durante la interacción con la interfaz gráfica.

Además, durante el proceso de carga de archivos, el sistema verifica que la información ingresada corresponda al formato esperado por la aplicación. En caso de detectar errores estructurales, incompatibilidades o problemas durante la lectura de datos, el programa informa adecuadamente al usuario, evitando incorporar información inválida dentro de las estructuras principales del backend.

Finalmente, también se implementaron restricciones asociadas a la navegación dentro de ciertas ventanas del sistema. Por ejemplo, durante la revisión de ejercicios pertenecientes a una rutina, el programa deshabilita dinámicamente determinadas opciones de navegación cuando estas podrían generar desplazamientos inválidos dentro de la colección correspondiente, contribuyendo así a mantener una interacción más coherente y controlada con la interfaz gráfica.

4. Conclusión Técnica

La implementación de la presente solución permitió desarrollar un sistema estructurado y funcional capaz de integrar programación orientada a objetos, programación orientada a eventos y diseño modular dentro de una aplicación gráfica desarrollada en Java Swing. A través de la separación entre frontend y backend, fue posible distribuir adecuadamente las responsabilidades asociadas a la interfaz gráfica, procesamiento lógico y administración de datos, favoreciendo una arquitectura más organizada y mantenible.

Desde el punto de vista técnico, el desarrollo del sistema permitió comprender de manera práctica la importancia de la comunicación entre componentes, especialmente en aplicaciones donde la interacción del usuario ocurre constantemente mediante eventos asociados a ventanas y elementos gráficos. Asimismo, la utilización de clases controladoras y estructuras de datos dinámicas permitió implementar reglas de negocio complejas manteniendo una lógica centralizada y coherente.

Otro aspecto relevante corresponde a la incorporación de mecanismos de validación orientados tanto al tratamiento de archivos externos como al control de datos ingresados por el usuario. Estas medidas permitieron fortalecer la robustez del sistema y mejorar la integridad de la información procesada durante la ejecución del programa.

Finalmente, la solución desarrollada logra representar adecuadamente los requerimientos planteados inicialmente, incorporando además mejoras adicionales relacionadas con restricciones lógicas y control de consistencia. Como resultado, se obtuvo una aplicación capaz de gestionar rutinas deportivas de manera organizada, validada y suficientemente flexible para responder correctamente a distintos escenarios de uso.